OASIS INFOBYTE — SIP Task List

> *"Learning is the only thing the mind never exhausts, never fears, and never regrets." — Leonardo da Vinci*

📌 QUICK NAVIGATION MENU

Jump straight to your track by finding your specific domain. *Press* ***Ctrl+F*** *(Windows/Linux) or* ***Cmd+F*** *(Mac) and type your track name exactly as listed below to jump directly to your assigned task cards.*

- 🌐 ***Web Development & Designing***
- 📱 ***Android App Development***
- ☕ ***Java Development***
- 🎨 ***Graphics Design***
- 📊 ***Data Science***
- 🐍 ***Python Programming***
- 📈 ***Data Analytics***
- 🔐 ***Security Analyst***

> *Each track section in Section 3 is separated by a clear horizontal rule. You only need to read your assigned section.*

SECTION 1 — MASTER ONBOARDING CHECKLIST

Complete every item below ***before*** *submitting any task. This checklist applies to* ***all tracks and all interns*** *without exception.*

1.1 · Profile & Setup (Do This First)

- *[ ]* ***Complete your LinkedIn profile.*** *Watch the LinkedIn Profile Improvement & Resume Building Tutorial Click here before starting any project work.*
- *[ ]* ***Review your Placement Support Materials.*** *Click here Read them fully — they support your post-internship job search.*
- *[ ]* ***Create your GitHub repository.*** *Name it exactly:* `OIBSIP` *(all caps). This single repository will hold all of your task submissions across the internship. Do not create separate repos per task.*

- *[ ] **Connect your GitHub account** and ensure your profile is public so evaluators can access your work.*

1.2 · Per-Task Workflow (Repeat for Every Task)

*Follow these steps **in order** for each task you complete:*

- *[ ] **Step 1 — Review.** Read the Task Card for your chosen task in Section 3 of this guide carefully. Understand the full Feature Checklist before writing a single line of code or making a single design.*

- *[ ] **Step 2 — Build.** Develop your project. Refer to the Tech Stack and Architecture requirements listed in the Task Card.*

*[ ] **Step 3 — Push to GitHub.** Commit your completed project to your `OIBSIP` repository. Use the **strict folder naming format** below — no exceptions:*

OIBSIP/[TrackName]-[Level/Task]-[ProjectName]/

- ***Examples:***
  - *`OIBSIP/WebDev-L1-LandingPage/`*
  - *`OIBSIP/Python-Task2-BMICalculator/`*
  - *`OIBSIP/DataAnalytics-L1-EDARetailSales/`*
  - *`OIBSIP/CyberSecurity-Task4-NetworkThreatsReport/`*
- *Each folder must contain your source code, a `README.md` describing the project, and any relevant screenshots or output files.*

- *[ ] **Step 4 — Record a Demo Video.** Create a screen-recorded walkthrough of your completed project. **The video must begin with a clear 2-second static title card or text overlay displaying: (1) your Full Name, (2) your Assigned Track, and (3) the Task Title.** After the title card, the video must show the app, site, or report functioning end-to-end — not just static screenshots. For report-based tasks (e.g., Cyber Security research tasks), narrate your document on screen.*

- *[ ] **Step 5 — Post to LinkedIn.** Upload or link your demo video in a LinkedIn post. Tag **Oasis Infobyte** in the post. Every post must include the hashtag **`#oasisinfobyte`**. Add relevant domain hashtags as appropriate (e.g., `#webdevelopment`, `#datascience`, `#androiddev`, `#python`, `#cybersecurity`, `#internship`).*

- *[ ] **Step 6 — Peer Evaluation.** Watch the LinkedIn demo videos posted by **at least two other interns** from your cohort. Leave a **substantive comment** on each (not*

just "Great work!" — engage with their implementation, ask a question, or offer a constructive observation). This is a graded requirement.

- *[ ] **Step 7 — Submit.** Submit your task using the **Task Submission Form** that has been shared with you via orientation email / offer letter email . Include your `OIBSIP` GitHub repository link in the form.*

  *📝 **Note:** The submission form will open on the date mentioned in the form shared via email midway through your cohort batch. Do not wait for the form to open before starting your project. Continue building and pushing your code to GitHub in the meantime. By the time the form opens, your work should already be ready for submission.*
-
- *[ ] **Step 8 — Wait for Evaluation.** After submission, await your evaluation takes around 2 weeks post the last date of submission. Upon successful completion, you will receive your **Completion Certificate** and, where applicable, an **Appreciation & Recommendation Letter**.*

1.3 · Key Reminders

*⚠️ **GitHub Repo Name:** Must be exactly `OIBSIP` (all caps) — any other name may cause your submission to be missed by evaluators.*

*⚠️ **Folder Naming:** Follow the strict `OIBSIP/[TrackName]-[Level/Task]-[ProjectName]/` format — inconsistent naming delays evaluation.*

*⚠️ **Video Title Card:** The first 2 seconds must show your **Full Name + Track + Task Title** as a static frame — videos without this will be returned for correction.*

*⚠️ **Peer Evaluation:** Substantive comments on at least **two** fellow intern videos are required for completion — generic comments do not qualify.*

*⚠️ **Completion Threshold:** Check Section 2 below to confirm exactly how many tasks your track requires. Do not under-submit.*

SECTION 2 — DOMAIN TRACK MATRIX

Refer to this table to understand your exact completion target before starting work. Find your track, note the minimum requirement, and begin.

Track	Tasks Available	Minimum to Complete	Notes
🌐 *Web Development & Design*	*7 (across 3 Levels)*	***Any 1 complete Level***	*Choose Level 1 (3 tasks), Level 2 (4 tasks), OR Level 3 (1 task). Complete ALL tasks within your chosen Level.* ⚠️ ***Level 3 Warning: Requires robust full-stack, backend & database knowledge. Not recommended for beginners.***
📱 *Android App Development*	*5 tasks*	***At least 3 tasks***	*Any 3 of the 5 listed tasks.*
☕ *Java Development*	*5 tasks*	***At least 2 tasks***	*Any 2 of the 5 listed tasks.*
🎨 *Graphics Design*	*5 tasks*	***At least 4 tasks***	*Any 4 of the 5 listed tasks.*
📊 *Data Science*	*5 tasks*	***At least 3 tasks***	*Any 3 of the 5 listed tasks.*
🐍 *Python Programming*	*5 tasks*	***At least 3 tasks***	*Any 3 of the 5 listed tasks. Each task has a Beginner and Advanced tier — choose your level of depth.*

📈 *Data Analytics*	*7 tasks (across 2 Levels)*	***At least 3 tasks from Level 1 or Level 2***	*Complete at least 3 tasks from either level. To be eligible for LOR, complete the maximum tasks from both levels.*
🔐 *Cyber Security*	*10 tasks (across 3 difficulty tiers)*	***At least 4 tasks***	*Any 4 of the 10 listed tasks. Mix of practical tasks (requiring a demo video) and research reports (no video required — submit as a `.md` file).*

> 💡 ***Pro tip:*** *Even if your track only requires 2–3 tasks, completing additional tasks significantly strengthens your portfolio and increases your eligibility for a Recommendation Letter and Job Opportunity referral.*

SECTION 3 — STANDARDISED TASK CARDS

Each card follows this format: ***Track → Level/Number → Task Title → Objective → Tech Stack → Feature Checklist → Self-Sourcing Guideline.***

Use the ***Quick Navigation Menu*** *at the top of this document (Ctrl+F / Cmd+F) to jump directly to your track. Every track is separated by a clear horizontal rule.*

🌐 TRACK: WEB DEVELOPMENT & DESIGNING

> ***Completion Rule:*** *Complete ALL tasks within* ***one chosen Level*** *(Level 1, 2, or 3). ⚠️ Level 3 requires full-stack backend and database knowledge. Choose your level based on your current skill set.*

LEVEL 1

TASK 1 · Landing Page

Objective: *Build a visually polished static landing page for a product, service, or brand of your choice. This task establishes foundational HTML/CSS layout skills.*

Tech Stack: *HTML5, CSS3 (no JavaScript required)*

Feature Checklist:

- *[ ] Fixed or sticky navigation bar with at least 3 navigation links*
- *[ ] Hero section with a headline, subheadline, and a call-to-action button*
- *[ ] At least 2 distinct content sections (e.g., Features, About, Testimonials)*
- *[ ] Footer with placeholder contact/social links*
- *[ ] Consistent colour palette applied across all sections*
- *[ ] Responsive layout — page must not break at mobile screen widths (use CSS Flexbox or Grid)*
- *[ ] No element overlap — all padding, margin, and box-sizing must be intentionally set*
- *[ ] Clean, readable typography with at least 2 font sizes used (heading vs. body)*

Self-Sourcing Guideline: *Search* ***"landing page HTML CSS tutorial beginner"*** *on YouTube for walkthrough inspiration. Browse* ***Dribbble*** *or* ***Awwwards*** *(search "landing page design") to gather visual inspiration before you build. Do not copy — use them to understand layout principles.*

TASK 2 · Personal Portfolio

Objective: *Create a personal portfolio website that showcases your skills, projects, and contact information — effectively your digital résumé.*

Tech Stack: *HTML5, CSS3 (JavaScript optional for enhancements)*

Feature Checklist:

- *[ ] Profile/hero section: your name, role title, and a professional photo or avatar placeholder*

- *[ ] About Me section: 2–3 sentences describing your background and interests*
- *[ ] Skills section: a visual grid or list of your technical skills*
- *[ ] Projects section: at least 2 placeholder project cards (title, description, GitHub link placeholder)*
- *[ ] Contact section: name, email, and optionally LinkedIn/GitHub icons*
- *[ ] Smooth scroll navigation between sections*
- *[ ] Consistent branding (colour scheme, font family) throughout*
- *[ ] Fully responsive — usable on both desktop and mobile*

Self-Sourcing Guideline: *Search* ***"personal portfolio website HTML CSS GitHub"*** *to find open-source portfolio templates to study (not copy). Use* ***Google Fonts*** *(fonts.google.com) for free typography. Search* ***"portfolio design inspiration 2024"*** *on Pinterest or Dribbble for layout ideas.*

TASK 3 · Temperature Converter Website

Objective: *Build an interactive web tool that converts temperature values between Celsius, Fahrenheit, and optionally Kelvin, with real-time input validation.*

Tech Stack: *HTML5, CSS3, JavaScript (Vanilla)*

Feature Checklist:

- *[ ] Numeric input field for the temperature value (validate: reject non-numeric input with an error message)*
- *[ ] Unit selector: dropdown menu or radio buttons allowing the user to choose the* ***input*** *unit (Celsius / Fahrenheit / Kelvin)*
- *[ ] Unit selector: separate control or auto-conversion showing all output units simultaneously*
- *[ ] Convert button that triggers the calculation on click*
- *[ ] Result display area showing the converted value(s) with correct unit labels*
- *[ ] Edge case handling: display a user-friendly message for absolute zero violations (e.g., input below −273.15°C)*
- *[ ] Clean, centred UI layout with clear labels*

Self-Sourcing Guideline: *Search* ***"temperature converter JavaScript tutorial"*** *on YouTube to understand the conversion formulas (C to F: multiply by 9/5, add 32; F to C: subtract 32, multiply by 5/9; Kelvin: add/subtract 273.15). Study the MDN Web Docs article on* ***"HTML input validation"*** *for the input-checking approach.*

LEVEL 2

TASK 1 · Calculator

Objective: *Build a fully functional browser-based calculator capable of performing basic arithmetic operations with a user-friendly button interface.*

Tech Stack: *HTML5, CSS3, JavaScript (Vanilla)*

Feature Checklist:

- *[ ] Display screen that shows the current input and result*
- *[ ] Numeric buttons (0–9) and a decimal point button*
- *[ ] Operator buttons: addition (+), subtraction (−), multiplication (×), division (÷)*
- *[ ] Equals (=) button to evaluate the expression*
- *[ ] Clear (C) button to reset the display*
- *[ ] Backspace/Delete button to remove the last entered character*
- *[ ] Prevent division-by-zero: display an error message instead of crashing*
- *[ ] Operator chaining: allow sequential operations (e.g., 5 + 3 × 2) without requiring a full reset*
- *[ ] CSS Grid used for button layout alignment*
- *[ ] Event listeners on all buttons (no inline `onclick` attributes in HTML)*

Self-Sourcing Guideline: *Search* ***"vanilla JavaScript calculator project tutorial"*** *on YouTube. Reference MDN Docs for `addEventListener`, `switch` statements, and `parseFloat()`. Avoid using `eval()` — build your own logic using variables and conditional statements.*

TASK 2 · Tribute Page

Objective: *Design and build a visually engaging tribute page dedicated to a historical figure, scientist, artist, or public figure you admire.*

Tech Stack: *HTML5, CSS3 (JavaScript optional)*

Feature Checklist:

- *[ ] Page title with the subject's name and a one-line tagline*
- *[ ] A prominent image (use a royalty-free image — source from* ***Unsplash*** *or* ***Wikimedia Commons****)*
- *[ ] A biography or tribute section: at least 3–4 paragraphs of original written content*
- *[ ] A timeline or key achievements section (ordered list or styled cards)*
- *[ ] A quote block: a notable quote from or about the subject, styled distinctly*
- *[ ] At least 2 different background colours used across sections*
- *[ ] At least 2 font styles explored (e.g., serif for headings, sans-serif for body)*
- *[ ] Responsive layout*

Self-Sourcing Guideline: *Use* ***Wikipedia*** *or* ***Britannica*** *to research your subject for factual content (paraphrase — do not copy). Source images from* ***Unsplash.com*** *(search by person's name or era) or* ***Wikimedia Commons*** *for public domain images. Search* ***"HTML CSS tribute page freeCodeCamp"*** *for structural guidance.*

TASK 3 · To-Do Web App

Objective: *Develop an interactive to-do list application that allows users to manage daily tasks with add, complete, edit, and delete functionality, organised into pending and completed lists.*

Tech Stack: *HTML5, CSS3, JavaScript (Vanilla or with a small library like Alpine.js)*

Feature Checklist:

- *[ ] Input field + "Add Task" button to create new tasks*
- *[ ] Newly added tasks appear immediately in the **Pending Tasks** list*
- *[ ] Each task has a "Mark Complete" toggle — completed tasks move to the **Completed Tasks** list*
- *[ ] Each task has an **Edit** button that allows the task text to be modified inline*
- *[ ] Each task has a **Delete** button that permanently removes it from either list*
- *[ ] Task count indicators: "X pending" and "Y completed" displayed above each list*
- *[ ] (Bonus) Timestamp displayed on each task showing when it was added and/or completed*
- *[ ] (Bonus) Tasks persist across page refreshes using `localStorage`*
- *[ ] Empty state messaging: display a friendly message when a list has no items*

Self-Sourcing Guideline: *Search **"JavaScript to-do list app tutorial DOM manipulation"** to understand the core pattern. For `localStorage` persistence, search **"localStorage JavaScript beginners guide MDN"**. Look at GitHub repositories with the topic **"todo-app vanilla-js"** for architecture inspiration (read — don't copy code).*

TASK 4 · Login Authentication System

Objective: *Build a simple client-side (or full-stack) authentication system featuring user registration, login validation, and access to a protected page.*

Tech Stack: *Choose one approach — (A) Front-end only: HTML/CSS/JavaScript with `localStorage`, (B) Full-stack: Node.js + Express + a simple JSON or SQLite store, or (C) Python Flask with session management.*

Feature Checklist:

- *[ ] **Registration page:** fields for username/email and password, with a "Register" button*
- *[ ] Password validation on registration: minimum 8 characters, at least 1 number*
- *[ ] Duplicate username/email check — display an error if the user already exists*
- *[ ] **Login page:** fields for username/email and password, with a "Login" button*
- *[ ] Incorrect credential handling: display a clear error message (do not reveal which field is wrong)*

- *[ ] **Protected/Dashboard page:** only accessible after successful login; redirect to login page if accessed directly without a session*
- *[ ] **Logout button** on the dashboard that clears the session/`localStorage` and redirects to login*
- *[ ] Passwords must not be stored in plain text — use a basic hashing approach (e.g., `bcrypt` for Node/Python, or a SHA-256 approach for client-side)*
- *[ ] Basic form validation on both pages (no empty submissions)*

***Self-Sourcing Guideline:** Search **"login authentication system JavaScript localStorage tutorial"** for the front-end approach, or **"Node.js Express login authentication bcrypt tutorial"** for the full-stack approach. Reference the MDN article on **"HTTP cookies and sessions"** for understanding session management concepts.*

LEVEL 3

> ⚠️ ***Warning:** Level 3 is a single, complex full-stack project. It requires solid knowledge of React, Node.js, MongoDB, and API integration. Do not choose this level unless you are comfortable with all of these technologies.*

TASK 1 · Pizza Delivery Full-Stack Application

***Objective:** Build a production-grade, full-stack pizza ordering and inventory management platform with separate Admin and User roles, real-time order tracking, payment integration, and automated stock notifications.*

***Tech Stack:** React.js (frontend), Node.js + Express.js (backend), MongoDB (database), Razorpay (payment — test mode)*

Feature Checklist:

User Side:

- *[ ] User registration with email verification*
- *[ ] User login with JWT-based authorisation*
- *[ ] Forgot password flow (email reset link)*
- *[ ] Dashboard displaying available pizza varieties*
- *[ ] Custom pizza builder flow:*
  - *[ ] Step 1: Choose a pizza base (5 options)*
  - *[ ] Step 2: Choose a sauce (5 options)*
  - *[ ] Step 3: Choose a cheese type*
  - *[ ] Step 4: Choose vegetables (multiple select)*
- *[ ] Order summary page before payment*
- *[ ] Razorpay checkout integration (test mode — clicking "Success" confirms the order)*

- *[ ] Real-time order status display on user dashboard (Order Received → In Kitchen → Sent to Delivery)*

Admin Side:

- *[ ] Separate admin login (not accessible from the user registration flow)*
- *[ ] Inventory dashboard showing current stock of: pizza bases, sauces, cheeses, vegetables*
- *[ ] Stock automatically decremented after each order*
- *[ ] Manual stock update capability for each inventory item*
- *[ ] Automated email notification to admin when any inventory item falls below a configurable threshold (e.g., pizza bases < 20 units) — implement using a scheduled job (e.g., `node-cron`)*
- *[ ] Order management panel: view all incoming orders, update status for each order*
- *[ ] Status change reflected in real-time on the user's dashboard (use polling or WebSockets)*

Self-Sourcing Guideline: *Search **"MERN stack full project tutorial 2024"** on YouTube for end-to-end architecture guidance. For Razorpay test integration, search **"Razorpay test mode Node.js integration"** — Razorpay's official developer documentation has a clear quickstart guide. For email notifications, search **"nodemailer Node.js email tutorial"** and **"node-cron scheduled jobs"**.*

📱 TRACK: ANDROID APP DEVELOPMENT

Completion Rule: *Complete* ***at least 3*** *of the 5 tasks below.*

TASK 1 · Unit Converter Application

Objective: *Build an Android app that converts values between common units of measurement (length, weight, volume, etc.) based on user input.*

Tech Stack: *Android Studio, Java, XML*

Feature Checklist:

- *[ ] Input field for the numeric value to be converted*
- *[ ] Dropdown (Spinner) to select the* ***source unit*** *(e.g., centimetres, grams, litres)*
- *[ ] Dropdown (Spinner) to select the* ***target unit***
- *[ ] Convert button that computes and displays the result*
- *[ ] Result TextView displaying the converted value with unit label*
- *[ ] Support for at least* ***3 measurement categories****: length, weight, and one more of your choice (e.g., temperature, volume, speed)*
- *[ ] Input validation: display a Toast message if the input field is empty or non-numeric*
- *[ ] Category selector allowing the user to switch between measurement types (resets unit dropdowns accordingly)*

Self-Sourcing Guideline: *Search* ***"Android unit converter app tutorial Java XML"*** *on YouTube. Reference the* ***Android Developer documentation*** *(developer.android.com) for* `Spinner`*,* `EditText`*, and* `Toast` *usage. For conversion formulas, a simple internet search (e.g., "cm to inches formula") is sufficient.*

TASK 2 · To-Do App with Login

Objective: *Build a secure task management app where users register and log in before accessing their personal to-do lists. Task data must persist locally using a database.*

Tech Stack: *Android Studio, Java, XML, SQLite*

Feature Checklist:

- *[ ]* ***Login screen:*** *email/username and password fields + Login button*
- *[ ]* ***Sign-up screen:*** *fields for name, email, password + Register button*
- *[ ] Passwords stored using basic hashing in SQLite (do not store plain text)*
- *[ ]* ***Logout*** *button in the main app that returns to the login screen and clears the session*
- *[ ]* ***Task list screen:*** *displays all tasks for the logged-in user*

- *[ ] "Add Task" button opens a dialog or new screen to enter a task name and optional notes*
- *[ ] Each task can be marked as **complete** (shown with a strikethrough or moved to a completed section)*
- *[ ] Each task can be **deleted** permanently*
- *[ ] SQLite database stores tasks linked to each user account (tasks are user-specific)*
- *[ ] Empty state: display a friendly message when the task list is empty*

***Self-Sourcing Guideline:** Search **"Android SQLite login register Java tutorial"** on YouTube for a complete walkthrough. Reference the Android Developer docs for **SQLiteOpenHelper** patterns. For understanding password hashing without external libraries, search **"Android MD5 SHA password hashing Java"**.*

TASK 3 · Calculator

***Objective:** Build a clean, fully functional Android calculator app that handles basic arithmetic operations with a responsive button grid interface.*

***Tech Stack:** Android Studio, Java, XML*

Feature Checklist:

- *[ ] Display TextView showing current input and result*
- *[ ] Number buttons (0–9) and decimal point*
- *[ ] Operator buttons: +, −, ×, ÷*
- *[ ] Equals (=) button to evaluate*
- *[ ] Clear (C) button to reset*
- *[ ] Backspace button to delete the last character*
- *[ ] Division-by-zero error handling: show "Error" in the display*
- *[ ] Button grid laid out using `GridLayout` or `ConstraintLayout` in XML*
- *[ ] No crashes on rapid/repeated button taps*

***Self-Sourcing Guideline:** Search **"Android calculator app Java tutorial beginners"** on YouTube. Reference Android Developer docs for `GridLayout`, `TextView`, and `OnClickListener`. Study how to use a `StringBuilder` to build the expression string before evaluating.*

TASK 4 · Quiz Application

***Objective:** Build a multiple-choice quiz app on a topic of your choice (e.g., general knowledge, science, history). Users answer questions one at a time and see their final score at the end.*

***Tech Stack:** Android Studio, Java, XML*

Feature Checklist:

- *[ ] Welcome screen with a Start button*
- *[ ] Question screen showing: the question text, 4 answer options as Radio Buttons or styled Button views, and a question counter (e.g., "Question 3 of 10")*
- *[ ] At least **10 questions** hardcoded or loaded from a local JSON/array*
- *[ ] Immediate feedback on answer selection: correct answer highlighted green, wrong answer highlighted red*
- *[ ] "Next" button to advance to the following question*
- *[ ] Score tracking throughout the quiz*
- *[ ] Results screen at the end showing: total score, number correct, number incorrect, and a "Restart Quiz" button*
- *[ ] Questions ideally presented in a shuffled order each time*

Self-Sourcing Guideline: *Search **"Android quiz app Java tutorial"** on YouTube. For question content, use **OpenTriviaDB (opentdb.com)** — it offers a free API with thousands of trivia questions, or you can hardcode 10 questions manually. Reference Android Developer docs for `RadioGroup`, `Intent` for navigation between Activities, and `Bundle` for passing data.*

TASK 5 · Stopwatch

Objective: *Build a functional stopwatch app with start, stop/pause, and reset controls that accurately tracks elapsed time.*

Tech Stack: *Android Studio, Java, XML*

Feature Checklist:

- *[ ] Large, clearly visible time display in `HH:MM:SS` format (or `MM:SS:ms` for millisecond precision)*
- *[ ] **Start** button: begins the timer from 0 (or from paused state)*
- *[ ] **Stop/Pause** button: freezes the timer at the current elapsed time*
- *[ ] **Reset** button: stops the timer and resets the display to `00:00:00`*
- *[ ] Buttons visually change state (e.g., Start becomes greyed out once running; Stop becomes active)*
- *[ ] Timer continues running correctly if the user navigates away and returns (handle `onPause`/`onResume` lifecycle)*
- *[ ] (Bonus) Lap functionality: a "Lap" button records the current time to a scrollable list below the display*

Self-Sourcing Guideline: *Search **"Android stopwatch app Java Handler tutorial"**. The standard implementation uses Android's `Handler` and `Runnable` to update the UI thread every 10–100 milliseconds. Reference the Android Developer docs section on **"Processes and threads"** and **"Activity Lifecycle"** to correctly handle pause/resume.*

☕ TRACK: JAVA DEVELOPMENT

Completion Rule: *Complete* ***at least 2*** *of the 5 tasks below.*

TASK 1 · Online Reservation System

Objective: *Build a GUI-based train/transport reservation system where users can log in, book tickets, and cancel bookings using a PNR number.*

Tech Stack: *Java (Swing or JavaFX for GUI), JDBC, MySQL or SQLite*

Feature Checklist:

- *[ ]* ***Login Form:*** *username + password fields; access denied for invalid credentials*
- *[ ]* ***Reservation Form:*** *fields for passenger name, train number, train name (auto-populated from train number), class type, date of journey, source station, destination station*
- *[ ] Insert/Book button that saves the reservation to the database and generates a* ***PNR number*** *(auto-generated unique ID)*
- *[ ] Confirmation dialog showing booking details after successful reservation*
- *[ ]* ***Cancellation Form:*** *PNR number input field + Fetch button that retrieves and displays the full booking details*
- *[ ] Confirm cancellation button with an "Are you sure?" dialog; removes the booking from the database on confirmation*
- *[ ] Basic input validation: no empty required fields, valid date format, numeric train number*

Self-Sourcing Guideline: *Search* ***"Java Swing JDBC login form tutorial"*** *on YouTube for GUI and database connection patterns. For database setup, search* ***"SQLite Java JDBC Maven setup"*** *(SQLite requires no separate server). Reference the official Java documentation for* `JFrame`*,* `JTextField`*,* `JComboBox`*, and* `PreparedStatement` *for SQL injection prevention.*

TASK 2 · Number Guessing Game

Objective: *Build a console or GUI-based game where the computer generates a random number and the user attempts to guess it, receiving higher/lower hints until correct.*

Tech Stack: *Java (console application or Swing GUI)*

Feature Checklist:

- *[ ] System generates a random number in the range **1 to 100** at the start of each round*
- *[ ] User is prompted to enter a guess (input via Scanner for console, or JTextField for GUI)*
- *[ ] After each guess, display one of three responses: "Too High!", "Too Low!", or "Correct!"*
- *[ ] Attempt counter visible to the user throughout the game*
- *[ ] Maximum attempts limit (e.g., 7 attempts) — game ends with a "You Lost!" message if limit is reached, revealing the number*
- *[ ] "Play Again" option after each round (yes/no prompt or button)*
- *[ ] Score tracking across multiple rounds: display "Round X — guessed in Y attempts" summary*
- *[ ] (Bonus) Difficulty levels: Easy (1–50, 10 attempts), Medium (1–100, 7 attempts), Hard (1–200, 5 attempts)*

***Self-Sourcing Guideline:** Search **"Java number guessing game tutorial beginners"** on YouTube. Core Java concepts needed: `java.util.Random`, `Scanner`, `while` loops, `if-else` statements. For the GUI version, search **"Java Swing simple game tutorial"**.*

TASK 3 · ATM Interface

***Objective:** Build a console-based simulation of an ATM machine that allows users to authenticate with a PIN and perform standard banking transactions.*

***Tech Stack:** Java (console application, Object-Oriented design with multiple classes)*

Feature Checklist:

- *[ ] Startup prompt for **User ID** and **PIN**; deny access after 3 incorrect attempts*
- *[ ] Main menu displayed after successful login with the following options:*
  - *[ ] **1. Transaction History** — display a log of all past transactions in the current session*
  - *[ ] **2. Withdraw** — prompt for amount; validate sufficient balance; update balance; log transaction*
  - *[ ] **3. Deposit** — prompt for amount; update balance; log transaction*
  - *[ ] **4. Transfer** — prompt for recipient account ID and amount; validate balance; update both accounts; log transaction*
  - *[ ] **5. Quit** — display a goodbye message and exit*
- *[ ] Balance check before any withdrawal or transfer; display "Insufficient Funds" if balance is too low*
- *[ ] All transactions stored in an `ArrayList` and displayed clearly in Transaction History*
- *[ ] At least 5 distinct Java classes: `ATM`, `Account`, `Transaction`, `Bank`, `Main`*

***Self-Sourcing Guideline:** Search **"Java ATM machine project OOP tutorial"** on YouTube. This project is a classic OOP exercise — study how to use **encapsulation** (private fields +*

*getters/setters), **ArrayList** for history, and **switch-case** for the menu system. Reference GeeksForGeeks or Javatpoint for Java OOP concepts.*

TASK 4 · Online Examination System

***Objective:** Build a GUI-based examination system where students log in, answer multiple-choice questions within a timer, and are auto-submitted when time runs out.*

***Tech Stack:** Java, Swing (GUI)*

Feature Checklist:

- *[ ] **Login screen:** username + password; on success, load the exam interface*
- *[ ] **Profile update screen:** allow the user to change their display name and password before starting*
- *[ ] **Exam screen:** displays one MCQ at a time with 4 radio button options*
- *[ ] Navigation: Next and Previous buttons to move between questions*
- *[ ] Countdown timer visible at all times (e.g., 30 minutes); auto-submits the exam when it reaches zero*
- *[ ] Manual submit button with a confirmation dialog*
- *[ ] **Result screen:** displays score (X out of Y), time taken, and a breakdown of correct/incorrect answers*
- *[ ] Session management: clicking the window's close button during an exam triggers a "Are you sure you want to quit?" dialog*
- *[ ] **Logout button** on the result screen that returns to the login screen*

***Self-Sourcing Guideline:** Search **"Java Swing MCQ exam timer tutorial"** on YouTube. Key components: `javax.swing.Timer` for the countdown, `ButtonGroup` + `JRadioButton` for MCQ options, `CardLayout` for switching between screens. Look for examples on **GitHub** by searching `"java-quiz-application swing"`.*

TASK 5 · Digital Library Management System

***Objective:** Build a web-based library system with separate Admin and User roles that manages a catalogue of books, handles issuing/returns, tracks fines, and supports advance bookings.*

***Tech Stack:** Java (Servlets/JSP or Spring Boot), MySQL or SQLite, HTML/CSS (frontend)*

Feature Checklist:

Admin Module:

- *[ ] Admin login with access to all system features*
- *[ ] Add new book: title, author, ISBN, category, quantity*
- *[ ] Edit or delete existing book records*

- *[ ] View all issued books and their due dates*
- *[ ] View and manage all registered member accounts*
- *[ ] Fine management: mark fines as paid*

User Module:

- *[ ] User registration and login*
- *[ ] Browse book catalogue by category*
- *[ ] Search for a specific book by title or author*
- *[ ] Issue a book (decrements available quantity; records due date)*
- *[ ] Return a book (increments available quantity)*
- *[ ] Fine generation: automatically calculated if return is overdue (e.g., ₹5 per day)*
- *[ ] Advance booking: reserve a book that is currently issued to another user*
- *[ ] Contact/query form (stores message in database; admin can view in the admin panel)*

Self-Sourcing Guideline: *Search* ***"Java Spring Boot library management system tutorial"*** *or* ***"Java Servlet JSP CRUD project"*** *on YouTube. For database schema design, search* ***"library management system database design MySQL"****. Reference Baeldung.com for Spring Boot patterns and MDN for any frontend HTML/CSS work.*

🎨 *TRACK: GRAPHICS DESIGN*

***Completion Rule:** Complete **at least 4** of the 5 tasks below.*

TASK 1 · Posters & Fliers

***Objective:** Design a set of at least 2 digital posters or promotional fliers for a real or fictional event, product launch, or awareness campaign.*

***Tech Stack / Tools:** Canva, Adobe Illustrator, Figma, or Adobe Photoshop (your choice)*

Feature Checklist:

- *[ ] At least **2 distinct poster/flier designs** (different events or campaign themes)*
- *[ ] Each design uses a defined colour palette (maximum 3–4 colours for visual cohesion)*
- *[ ] Clear **visual hierarchy**: headline, subheadline, body copy, and CTA (call-to-action) are visually distinguishable*
- *[ ] At least one custom graphic element per design (an icon, shape, or illustration — not just text on a background)*
- *[ ] Correct dimensions: A4 (210×297mm) for print posters, or 1080×1080px for social media square format*
- *[ ] All text is legible against its background (check contrast ratios)*
- *[ ] Exported as high-resolution PNG or PDF*

***Self-Sourcing Guideline:** Browse **Behance.net** (search "poster design" or "event flier") for professional inspiration. Use **Unsplash.com** for royalty-free background images. Use **Google Fonts** to explore type combinations. For Canva users, study **Canva's Design School** tutorials available free on their website.*

TASK 2 · Logo Design

***Objective:** Design a professional, minimalist logo for a fictional brand of your choice (tech startup, café, fitness app, etc.) that communicates the brand's identity clearly.*

***Tech Stack / Tools:** Adobe Illustrator (preferred for vector output), Figma, or Canva*

Feature Checklist:

- *[ ] Original logo concept — not a template modification*
- *[ ] Delivered in at least **3 variants**: full colour, monochrome (black), and reversed (white on dark)*
- *[ ] Vector format (SVG or AI source file) so the logo scales without quality loss*
- *[ ] Minimalist design philosophy (KISS principle: Keep It Simple)*
- *[ ] Typography used (if any) is intentional — custom letterforms or a well-chosen font pairing*

- *[ ] A brief **design rationale** (2–3 sentences) explaining your colour and symbol choices, included in your LinkedIn post*
- *[ ] Exported as PNG (transparent background) at minimum 1000×1000px*

***Self-Sourcing Guideline:** Search **"logo design process tutorial Adobe Illustrator"** on YouTube. For colour palette inspiration, use **Coolors.co** to generate harmonious palettes. Browse **Behance** (search "minimal logo design") and **LogoLounge.com** for professional examples. Read about **logo design principles** on the Nielsen Norman Group website.*

TASK 3 · Business Card Design

***Objective:** Design a professional double-sided business card for a fictional person and their company, demonstrating mastery of typography, whitespace, and branding consistency.*

***Tech Stack / Tools:** Adobe Illustrator, Figma, or Canva*

Feature Checklist:

- *[ ] **Front side:** Company logo, person's full name, and job title*
- *[ ] **Back side:** Contact details — phone, email, website, and 1–2 social media handles*
- *[ ] Standard business card dimensions: **85mm × 55mm** (3.5" × 2")*
- *[ ] Typography uses a maximum of **2 font families** (one for headings, one for body)*
- *[ ] Deliberate use of **whitespace** — not crowded*
- *[ ] Design is consistent with the brand identity from Task 2 (optional: use the logo you designed)*
- *[ ] Bleed area of 3mm included on all sides for print-readiness*
- *[ ] Exported as print-ready PDF and a preview PNG*

***Self-Sourcing Guideline:** Search **"business card design tutorial Figma/Canva/Illustrator"** on YouTube. For typography pairing, use **Fontpair.co**. Browse **Behance** (search "business card design 2024") for contemporary design trends. Reference Canva's free templates only for dimensional reference — your design must be original.*

TASK 4 · Infographic

***Objective:** Create a visually compelling infographic that presents data, a process, or a set of facts on a topic of your choice in a clear, engaging, and structured format.*

***Tech Stack / Tools:** Canva, Venngage, Adobe Illustrator, or Figma*

Feature Checklist:

- *[ ] A clear topic/title at the top*
- *[ ] At least **5 distinct data points or informational sections***

- *[ ] Minimum **3 types of visual elements**: icons, charts/graphs, illustrated diagrams, or progress bars*
- *[ ] Consistent colour scheme (maximum 4 colours)*
- *[ ] Visual flow that guides the reader's eye logically through the content (top-to-bottom or left-to-right)*
- *[ ] All statistics/data cited (note the source in small text at the bottom of the infographic)*
- *[ ] Standard infographic dimensions: 800×2000px (tall portrait format)*
- *[ ] Exported as PNG (high resolution, 150 DPI minimum)*

***Self-Sourcing Guideline:** For data and statistics, search **Google Scholar**, **Statista** (many free stats), or **Our World in Data** (ourworldindata.org — all free and citable). For infographic design tutorials, search **"how to make an infographic Canva tutorial"** or **"infographic design principles"** on YouTube. Browse **Visual.ly** for professional infographic examples.*

TASK 5 · Magazine or Brochure Layout

***Objective:** Design a 4–6 page magazine spread or tri-fold brochure layout for a fictional brand, publication, or organisation, demonstrating multi-page layout design and print media principles.*

***Tech Stack / Tools:** Adobe InDesign (preferred), Figma, or Canva (multi-page)*

Feature Checklist:

- *[ ] **Minimum 4 pages** (or both sides of a tri-fold brochure = 6 panels)*
- *[ ] Consistent **master page/template** elements across all pages (page numbers, header/footer, column grid)*
- *[ ] At least 2 full-bleed image pages (image extends to the edge of the page)*
- *[ ] Body copy uses a legible serif or sans-serif font at 10–12pt with appropriate line spacing*
- *[ ] At least one **pull quote** styled distinctly from body copy*
- *[ ] Images used are royalty-free (source from Unsplash.com or Pexels.com)*
- *[ ] Colour palette consistent with a defined brand identity*
- *[ ] Exported as a multi-page PDF*

***Self-Sourcing Guideline:** Search **"magazine layout design tutorial Adobe InDesign"** or **"brochure design Canva tutorial"** on YouTube. For layout and grid principles, read the article **"Grid Systems in Graphic Design"** — search it on Google. Browse **Behance** (search "magazine layout design" or "brochure design") for high-quality references.*

📊 TRACK: DATA SCIENCE

***Completion Rule:** Complete **at least 3** of the 5 tasks below.*

TASK 1 · Iris Flower Classification

***Objective:** Train a machine learning classification model to identify the species of an iris flower (Setosa, Versicolor, or Virginica) from its physical measurements.*

***Tech Stack:** Python, scikit-learn, pandas, matplotlib/seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Load the Iris dataset (available directly from `sklearn.datasets.load_iris()` — no download required)*
- *[ ] Perform **Exploratory Data Analysis (EDA)**: shape, dtypes, null value check, descriptive statistics*
- *[ ] Visualisations: pairplot or scatter matrix showing feature distributions by species; box plots for each feature*
- *[ ] **Feature selection** discussion: which features are most discriminative?*
- *[ ] Train/test split (typically 80/20) using `train_test_split`*
- *[ ] Train at least **2 different classifiers** (e.g., Logistic Regression, K-Nearest Neighbours, Decision Tree, Random Forest)*
- *[ ] Evaluate each model: accuracy score, confusion matrix, classification report (precision, recall, F1)*
- *[ ] Identify and declare the best-performing model with justification*
- *[ ] All code in a clean, commented Jupyter Notebook*

***Self-Sourcing Guideline:** The dataset is built into scikit-learn — no external download needed. Search **"Iris flower classification Python scikit-learn tutorial"** on YouTube for an end-to-end walkthrough. Reference the **scikit-learn User Guide** (scikit-learn.org) for model-specific documentation. For visualisations, search **"seaborn pairplot tutorial"**.*

TASK 2 · Unemployment Analysis with Python

***Objective:** Perform exploratory data analysis on unemployment data to uncover regional and temporal trends, with a focus on the impact of the COVID-19 pandemic on unemployment rates in India.*

***Tech Stack:** Python, pandas, matplotlib, seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Download a suitable dataset (see Self-Sourcing Guideline below)*
- *[ ] Data loading, shape inspection, null value check, and type conversion*

- *[ ] EDA: region-wise average unemployment rates, month-wise trends*
- *[ ] Time-series line chart: unemployment rate over time for at least 3 major states/regions*
- *[ ] Bar chart: top 10 states with highest average unemployment rate*
- *[ ] Heatmap: correlation between unemployment rate, employment rate, and labour participation rate*
- *[ ] Pre-COVID vs. post-COVID comparison (split data by date, calculate mean rates for each period)*
- *[ ] Written observations/markdown cells between each chart explaining what the data shows*
- *[ ] Clean, well-commented Jupyter Notebook*

Self-Sourcing Guideline: *Search **"unemployment rate India dataset"** on **Kaggle.com** (create a free account). The dataset titled **"Unemployment in India"** is publicly available there. Search **"pandas time series analysis tutorial"** and **"seaborn heatmap tutorial"** on YouTube for the core visualisation patterns.*

TASK 3 · Car Price Prediction with Machine Learning

Objective: *Build a regression model that predicts the selling price of a used car based on features such as brand, age, mileage, fuel type, and transmission.*

Tech Stack: *Python, pandas, scikit-learn, matplotlib, seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Download a suitable dataset (see Self-Sourcing Guideline below)*
- *[ ] Data cleaning: handle null values, remove duplicates, address inconsistent categorical values (e.g., "Petrol" vs. "petrol")*
- *[ ] Feature engineering: calculate **car age** from the year column; extract brand from the car name column*
- *[ ] EDA: distribution of selling prices, price vs. fuel type box plots, price vs. car age scatter plot*
- *[ ] Encode categorical variables (One-Hot Encoding or Label Encoding)*
- *[ ] Feature correlation heatmap*
- *[ ] Train/test split*
- *[ ] Train at least **2 regression models** (e.g., Linear Regression, Random Forest Regressor, Gradient Boosting)*
- *[ ] Evaluate models using: MAE, RMSE, and R^2 score*
- *[ ] Feature importance chart for your best-performing model*
- *[ ] All steps in a clean, commented Jupyter Notebook*

Self-Sourcing Guideline: *Search **"car price prediction dataset"** on **Kaggle.com**. A widely used dataset is **"Vehicle dataset from cardekho"** — it is publicly available and well-suited for this task. Search **"car price prediction Python machine learning tutorial"** on YouTube for guidance.*

TASK 4 · Email Spam Detection with Machine Learning

Objective: *Build a Natural Language Processing (NLP) binary classifier that distinguishes spam emails from legitimate (ham) emails.*

Tech Stack: *Python, pandas, scikit-learn (TF-IDF, Naive Bayes/SVM), NLTK or re, Jupyter Notebook*

Feature Checklist:

- *[ ] Download a suitable dataset (see Self-Sourcing Guideline below)*
- *[ ] Data loading and class distribution check (spam vs. ham counts and percentage)*
- *[ ] Text preprocessing pipeline: lowercase conversion, punctuation removal, stopword removal, optional stemming/lemmatization*
- *[ ] Feature extraction using **TF-IDF Vectorizer** (explain what TF-IDF measures in a markdown cell)*
- *[ ] Train/test split*
- *[ ] Train at least **2 classifiers**: Multinomial Naive Bayes (industry standard for text) + one alternative (e.g., Logistic Regression, SVM)*
- *[ ] Evaluation: accuracy, precision, recall, F1-score, confusion matrix*
- *[ ] Discussion: Why is **recall** particularly important for spam detection? (Answer in a markdown cell)*
- *[ ] (Bonus) WordCloud visualisations for spam words and ham words*
- *[ ] All steps in a clean, commented Jupyter Notebook*

Self-Sourcing Guideline: *Search **"SMS spam collection dataset"** on **Kaggle.com** or the **UCI Machine Learning Repository** (archive.ics.uci.edu) — both host this classic dataset for free. Search **"email spam detection Python NLP tutorial scikit-learn"** on YouTube. Reference the NLTK documentation (nltk.org) for text preprocessing.*

TASK 5 · Sales Prediction Using Python

Objective: *Build a regression model that predicts product sales based on advertising spend across different media channels (TV, Radio, Newspaper).*

Tech Stack: *Python, pandas, scikit-learn, matplotlib, seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Download a suitable dataset (see Self-Sourcing Guideline below)*
- *[ ] Data loading and EDA: null check, descriptive statistics, pairplot of all features*
- *[ ] Individual scatter plots: Sales vs. TV spend, Sales vs. Radio spend, Sales vs. Newspaper spend*
- *[ ] Correlation matrix heatmap*
- *[ ] Train/test split*
- *[ ] Train a **Linear Regression** model as the baseline*

- *[ ] Train at least one additional model (e.g., Random Forest Regressor, Polynomial Regression)*
- *[ ] Evaluate using: MAE, RMSE, R^2 score*
- *[ ] Residual plot for the best model (are errors randomly distributed or systematic?)*
- *[ ] Interpretation: which advertising channel has the highest impact on sales? (Answered using coefficients or feature importance)*
- *[ ] Clean, well-commented Jupyter Notebook*

Self-Sourcing Guideline: *Search* ***"advertising sales prediction dataset"*** *on* ***Kaggle.com****. The classic* ***"Advertising.csv"*** *dataset (TV, Radio, Newspaper → Sales) is widely available. Search* ***"sales prediction linear regression Python tutorial"*** *on YouTube. Reference the* ***scikit-learn documentation*** *for* `LinearRegression` *and evaluation metrics.*

🐍 *TRACK: PYTHON PROGRAMMING*

> ***Completion Rule:*** *Complete* ***at least 3*** *of the 5 tasks below. Each task has a* ***Beginner*** *and* ***Advanced*** *tier. Choose the depth that matches your current skill level — both tiers are acceptable for internship completion.*

TASK 1 · Voice Assistant

Objective: *Build a Python-based voice assistant that listens to spoken commands and responds with useful actions. Beginners implement core voice recognition; advanced builds add NLP, API integrations, and smart home features.*

Tech Stack: *Python,* `speech_recognition` *library,* `pyttsx3` *(text-to-speech),* `datetime`*,* `webbrowser` *— Advanced additionally uses:* `nltk` *or* `transformers`*, third-party APIs (weather, email),* `smtplib`

Feature Checklist — Beginner Tier:

- *[ ] Capture voice input using* `speech_recognition` *(microphone)*
- *[ ] Respond to "Hello" with a predefined greeting*
- *[ ] Tell the current time and date on request*
- *[ ] Perform a web search on a user-specified topic (open browser with query)*
- *[ ] Graceful error handling: if voice is not understood, ask the user to repeat*
- *[ ] Text-to-speech feedback using* `pyttsx3` *for all responses*

Feature Checklist — Advanced Tier (includes all Beginner features, plus):

- *[ ] Natural language understanding: parse intent from free-form spoken sentences (not just keyword matching)*

- *[ ] Send an email via voice command using `smtplib` (use a test/dummy email account)*
- *[ ] Set a timed reminder that triggers an audible alert after a specified duration*
- *[ ] Fetch and read out live weather updates using a weather API (e.g., OpenWeatherMap — free tier)*
- *[ ] Answer general knowledge questions using a QA API or local knowledge base*
- *[ ] Allow users to add custom commands via a config file or voice prompt*
- *[ ] Privacy consideration: document in your README which data is processed and how*

Self-Sourcing Guideline: *Search* ***"Python voice assistant tutorial speech_recognition pyttsx3"*** *on YouTube for the beginner approach. For the advanced tier, search* ***"Python NLP chatbot intent recognition tutorial"*** *and* ***"OpenWeatherMap API Python tutorial"****. Register for a free OpenWeatherMap API key at openweathermap.org. Reference the `speech_recognition` library documentation on PyPI for setup on different operating systems.*

TASK 2 · BMI Calculator

Objective: *Build a Python program that calculates a user's Body Mass Index (BMI) and classifies it into health categories. Beginners build a command-line tool; advanced builds a full GUI application with data persistence and trend visualisation.*

Tech Stack — Beginner: *Python, `input()`, basic arithmetic* ***Tech Stack — Advanced:*** *Python, `tkinter` or `PyQt5`, `matplotlib`, `sqlite3` or CSV file storage*

Feature Checklist — Beginner Tier:

- *[ ] Prompt user for weight (kg) and height (m) via command line*
- *[ ] Calculate BMI using the formula: `BMI = weight / (height²)`*
- *[ ] Classify result into standard categories: Underweight (< 18.5), Normal (18.5–24.9), Overweight (25–29.9), Obese (≥ 30)*
- *[ ] Display the BMI value rounded to 2 decimal places and the category*
- *[ ] Input validation: reject non-numeric input and negative values with a helpful error message*

Feature Checklist — Advanced Tier (includes all Beginner features, plus):

- *[ ] GUI window built with `tkinter` or `PyQt5` — no command line*
- *[ ] Input fields with labels for weight and height; a "Calculate" button*
- *[ ] Result displayed in the GUI with colour-coded feedback (e.g., green = normal, red = obese)*
- *[ ] Multi-user support: allow saving BMI records for different named users*
- *[ ] Historical records stored in an SQLite database or CSV file*
- *[ ] Graph view: display a line chart of a user's BMI trend over time using `matplotlib`*
- *[ ] Error handling for database read/write failures*

Self-Sourcing Guideline: *Search* ***"Python BMI calculator command line tutorial"*** *for the beginner approach. For the advanced tier, search* ***"Python tkinter GUI tutorial beginners"*** *and* ***"Python matplotlib line chart tutorial"****. Reference the official* `tkinter` *documentation (docs.python.org) for widget layouts. For SQLite, search* ***"Python sqlite3 tutorial CRUD"****.*

TASK 3 · Random Password Generator

Objective: *Build a Python tool that generates strong, random passwords based on user-defined criteria. Beginners build a command-line version; advanced builds a GUI with complexity controls and clipboard integration.*

Tech Stack — Beginner: *Python,* `random`*,* `string` ***Tech Stack — Advanced:*** *Python,* `secrets` *(cryptographically secure),* `tkinter` *or* `PyQt5`*,* `pyperclip`

Feature Checklist — Beginner Tier:

- *[ ] Prompt user to specify desired password length (minimum 8 characters enforced)*
- *[ ] Prompt user to choose which character types to include: uppercase letters, lowercase letters, numbers, symbols (at least 2 types must be selected)*
- *[ ] Generate and display a password matching all specified criteria*
- *[ ] Input validation: reject invalid lengths or no character types selected*
- *[ ] Option to generate another password without restarting the program*

Feature Checklist — Advanced Tier (includes all Beginner features, plus):

- *[ ] GUI window with sliders or spinboxes for length control and checkboxes for character type selection*
- *[ ] Use* `secrets` *module (not* `random`*) for cryptographically secure generation*
- *[ ] Password strength indicator: display a visual bar or label showing strength (Weak / Medium / Strong) based on length and character diversity*
- *[ ] Security rules enforced: generated password guaranteed to contain at least one character from each selected type*
- *[ ] "Copy to Clipboard" button using* `pyperclip` *— password copies automatically on generation*
- *[ ] Option to exclude ambiguous characters (e.g.,* `0`*,* `O`*,* `l`*,* `1`*) via a checkbox*
- *[ ] Generation history: display the last 5 generated passwords in the session (do not persist to file for security)*

Self-Sourcing Guideline: *Search* ***"Python password generator tutorial random string"*** *on YouTube for the beginner approach. For the advanced tier, search* ***"Python tkinter password generator GUI"*** *and* ***"Python secrets module vs random"****. Reference the Python documentation for* `secrets` *(docs.python.org/3/library/secrets.html) — use it instead of* `random` *for anything security-related. For clipboard integration, search* ***"pyperclip Python tutorial"****.*

TASK 4 · Basic Weather App

Objective: *Build a Python application that fetches and displays real-time weather data for a user-specified location using a weather API. Beginners build a command-line tool; advanced builds a graphical app with forecasts and visual elements.*

Tech Stack — Beginner: *Python, `requests`, `json`, OpenWeatherMap API (free tier)* ***Tech Stack — Advanced:*** *Python, `requests`, `tkinter` or `PyQt5`, `PIL`/`Pillow` for icons, OpenWeatherMap API*

Feature Checklist — Beginner Tier:

- *[ ] Prompt user to enter a city name or ZIP code*
- *[ ] Make an API call to OpenWeatherMap (or equivalent free API) and parse the JSON response*
- *[ ] Display: current temperature (°C and °F), humidity percentage, weather condition description (e.g., "Partly Cloudy"), wind speed*
- *[ ] Handle API errors gracefully: city not found, network timeout, invalid API key*
- *[ ] Input validation: reject empty city input*

Feature Checklist — Advanced Tier (includes all Beginner features, plus):

- *[ ] GUI window with a city input field, a "Get Weather" button, and a results panel*
- *[ ] Display weather icons corresponding to the current condition (use OpenWeatherMap icon URLs)*
- *[ ] Hourly forecast panel: show weather for the next 6 hours*
- *[ ] Daily forecast panel: show weather for the next 5 days*
- *[ ] Unit toggle: Celsius / Fahrenheit switch button*
- *[ ] (Bonus) Automatic location detection using the user's IP address via `ipinfo.io` API (free tier)*
- *[ ] Error messages shown inside the GUI (not terminal print statements)*

Self-Sourcing Guideline: *Register for a* ***free API key*** *at openweathermap.org — the free tier allows 60 calls/minute and is sufficient for this project. Search* ***"Python weather app OpenWeatherMap API tutorial"*** *on YouTube. Reference the OpenWeatherMap API documentation for JSON response structure. For the GUI version, search* ***"Python tkinter weather app tutorial"****.*

TASK 5 · Chat Application

Objective: *Build a real-time messaging application in Python. Beginners implement a two-user command-line chat via sockets; advanced builds a full GUI application with multiple rooms, authentication, and message history.*

Tech Stack — Beginner: *Python, `socket`, `threading`* ***Tech Stack — Advanced:*** *Python, `socket`/`asyncio` or Flask-SocketIO, `tkinter` or a web framework, `sqlite3` for message history*

Feature Checklist — Beginner Tier:

- *[ ] Server script that listens for incoming client connections*
- *[ ] Client script that connects to the server*
- *[ ] Real-time, bidirectional message exchange between two connected clients*
- *[ ] Messages displayed with a timestamp prefix (e.g., `[14:35] Alice: Hello`)*
- *[ ] Graceful disconnection handling: notify the other client when one disconnects*
- *[ ] Both scripts runnable on the same machine (using `localhost`)*

Feature Checklist — Advanced Tier (includes all Beginner features, plus):

- *[ ] GUI chat window built with `tkinter` or served as a web app using Flask*
- *[ ] User registration and login (username + password, stored in SQLite)*
- *[ ] Multiple chat rooms: users can create or join named rooms*
- *[ ] Message history: past messages in a room are loaded when a user joins*
- *[ ] Desktop or in-app notification for new messages when the window is not focused*
- *[ ] Emoji support: render common emoji shortcodes (e.g., `:smile:`) as Unicode characters*
- *[ ] End-to-end awareness: README must document how messages are stored and what is not encrypted (security transparency)*

Self-Sourcing Guideline: *Search **"Python socket programming tutorial beginners"** on YouTube for the core networking concepts. For the beginner version, search **"Python chat app socket threading tutorial"**. For the advanced GUI version, search **"Flask SocketIO real-time chat tutorial"** or **"Python tkinter chat application tutorial"**. Reference the official Python `socket` documentation (docs.python.org) for socket types and connection patterns.*

📈 TRACK: DATA ANALYTICS

> ***Completion Rule:*** *Complete* ***at least 3 tasks*** *from Level 1 or Level 2 combined. To be eligible for a Letter of Recommendation (LOR), complete the maximum number of tasks from* ***both*** *levels.*

LEVEL 1

TASK 1 · EDA on Retail Sales Data

Objective: *Perform a thorough Exploratory Data Analysis on a retail sales dataset to uncover patterns, customer behaviour trends, and actionable business insights.*

Tech Stack: *Python, pandas, matplotlib, seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Load dataset and perform initial inspection: shape, column dtypes, null value check*
- *[ ] Descriptive statistics: mean, median, mode, standard deviation for all numerical columns*
- *[ ] Time series analysis: plot monthly and quarterly sales trends using line charts*
- *[ ] Customer demographics analysis: distribution of customer age groups, gender breakdown*
- *[ ] Product analysis: top 10 best-selling products; revenue by product category (bar chart)*
- *[ ] Heatmap: correlation matrix between numerical variables*
- *[ ] At least one additional visualisation of your choice that reveals a non-obvious insight*
- *[ ] Markdown cells throughout the notebook with written observations after each chart*
- *[ ] Conclusion section: at least 3 specific, actionable business recommendations based on findings*

Self-Sourcing Guideline: *Search* ***"retail sales dataset"*** *on* ***Kaggle.com*** *— multiple suitable datasets are available (search terms: "superstore sales", "retail sales data", "e-commerce sales dataset"). Search* ***"pandas EDA tutorial retail sales"*** *and* ***"seaborn time series plot tutorial"*** *on YouTube for technique guidance.*

TASK 2 · Customer Segmentation Analysis

Objective: *Apply clustering algorithms to segment an e-commerce company's customer base into distinct groups based on purchasing behaviour, enabling targeted marketing strategies.*

Tech Stack: *Python, pandas, scikit-learn (KMeans), matplotlib, seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Load dataset and inspect structure; handle missing values and inconsistent data*
- *[ ] Descriptive statistics: average purchase value, purchase frequency, customer lifetime value*
- *[ ] Feature selection: identify 2–3 key behavioural features for clustering (e.g., Recency, Frequency, Monetary — RFM analysis)*
- *[ ] Data normalisation/standardisation before clustering (use `StandardScaler`)*
- *[ ] Apply **K-Means clustering** — use the Elbow Method to determine optimal number of clusters (K)*
- *[ ] Visualise clusters using scatter plots (at least 2 feature combinations)*
- *[ ] Profile each cluster: calculate mean feature values per cluster and describe the customer type*
- *[ ] Bar chart: number of customers per cluster*
- *[ ] Insights section: what marketing action would you recommend for each segment?*

Self-Sourcing Guideline: *Search **"customer segmentation RFM analysis Python tutorial"** on YouTube. For a dataset, search **"e-commerce customer data"** on **Kaggle.com** (the "Online Retail" dataset from UCI is a classic choice). Search **"KMeans elbow method scikit-learn tutorial"** for clustering implementation guidance.*

TASK 3 · Cleaning Data

Objective: *Demonstrate professional-level data cleaning skills by taking a deliberately messy dataset and systematically transforming it into a clean, analysis-ready dataset. Document every decision.*

Tech Stack: *Python, pandas, numpy, Jupyter Notebook*

Feature Checklist:

- *[ ] Load dataset and produce a "data quality report": count of nulls per column, duplicate rows, data type issues, value range anomalies*
- *[ ] **Missing data handling:** choose an appropriate strategy for each column (mean/median imputation, mode imputation, forward fill, or row deletion) and justify each choice in a markdown cell*
- *[ ] **Duplicate removal:** identify and remove duplicate rows; document how many were removed*
- *[ ] **Standardisation:** normalise inconsistent formatting (e.g., "Male"/"male"/"M" → "Male"; date formats → `datetime`)*
- *[ ] **Outlier detection:** use IQR method or Z-score to identify outliers in numeric columns; decide and document whether to cap, remove, or retain each*
- *[ ] **Data type correction:** ensure all columns have the correct dtype (e.g., dates as `datetime`, IDs as `string`, monetary values as `float`)*

- *[ ] Produce a "before vs. after" summary table: null count, duplicate count, row count, and dtype accuracy — before and after cleaning*
- *[ ] Save the cleaned dataset to a new CSV file*

Self-Sourcing Guideline: *For a messy dataset, search* ***"dirty dataset for data cleaning practice"*** *on* ***Kaggle.com*** *(search tags: "data cleaning"). Good options include the "Titanic dataset", "Housing dataset", or search specifically "messy data cleaning exercise". Search* ***"pandas data cleaning tutorial 2024"*** *on YouTube for comprehensive technique walkthroughs.*

TASK 4 · Sentiment Analysis

Objective: *Build a machine learning model that classifies the sentiment of text data (positive, negative, or neutral), providing insights into public opinion or customer feedback.*

Tech Stack: *Python, pandas, scikit-learn, NLTK or* `TextBlob`*, matplotlib/seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Load dataset and inspect class distribution (positive/negative/neutral counts)*
- *[ ] Text preprocessing pipeline: lowercase, punctuation removal, stopword removal, tokenisation, optional stemming/lemmatization*
- *[ ] Feature extraction: TF-IDF Vectorizer (explain its purpose in a markdown cell)*
- *[ ] Train/test split (80/20)*
- *[ ] Train at least* ***2 classifiers****: Naive Bayes + one of (SVM, Logistic Regression, or a simple LSTM if comfortable with deep learning)*
- *[ ] Evaluation: accuracy, precision, recall, F1-score, confusion matrix for each model*
- *[ ] Visualisation: bar chart of sentiment distribution in the dataset; WordCloud for each sentiment class*
- *[ ] Error analysis: show 5 examples the model misclassified and discuss why*
- *[ ] Conclusion: which model performed best and what real-world application could this serve?*

Self-Sourcing Guideline: *Search* ***"sentiment analysis dataset"*** *on* ***Kaggle.com*** *(good options: "Twitter Sentiment Analysis", "Amazon Product Reviews", "IMDB Movie Reviews"). Search* ***"Python sentiment analysis NLP tutorial scikit-learn"*** *on YouTube. Reference the NLTK documentation (nltk.org) for preprocessing and TextBlob's documentation for a simpler rule-based baseline.*

LEVEL 2

TASK 1 · Predicting House Prices with Linear Regression

Objective: *Build and evaluate a linear regression model that predicts house prices based on features such as area, location, number of rooms, and age. Develop end-to-end skills from data cleaning through to model interpretation.*

Tech Stack: *Python, pandas, scikit-learn, matplotlib, seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Load dataset and perform EDA: null check, descriptive statistics, distribution of the target variable (price)*
- *[ ] Feature selection discussion: identify which features are likely predictors; explain reasoning in a markdown cell*
- *[ ] Handle missing values and encode categorical features (One-Hot Encoding)*
- *[ ] Correlation heatmap: identify features most correlated with house price*
- *[ ] Train/test split (80/20)*
- *[ ] Train a **Linear Regression** model using scikit-learn*
- *[ ] Evaluate model using: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score*
- *[ ] Scatter plot: actual prices vs. predicted prices (the closer to a diagonal line, the better)*
- *[ ] Residual plot: check that residuals are randomly distributed (no systematic pattern)*
- *[ ] Coefficient analysis: which features have the highest positive/negative impact on price?*
- *[ ] (Bonus) Compare Linear Regression against a Ridge or Lasso regularised model*

Self-Sourcing Guideline: *Search **"house price prediction dataset"** on **Kaggle.com** (the "Ames Housing Dataset" and "House Prices: Advanced Regression Techniques" competition datasets are both excellent). Search **"linear regression house price prediction Python tutorial"** on YouTube. Reference the scikit-learn documentation for* `LinearRegression`*,* `mean_squared_error`*, and* `r2_score`*.*

TASK 2 · Wine Quality Prediction

Objective: *Train and compare multiple classification models to predict the quality score of wine (typically a scale of 3–8) based on its physicochemical properties such as acidity, density, and alcohol content.*

Tech Stack: *Python, pandas, numpy, scikit-learn (Random Forest, SGD, SVC), seaborn, matplotlib, Jupyter Notebook*

Feature Checklist:

- *[ ] Load the Wine Quality dataset and inspect structure; check class distribution of quality scores*
- *[ ] EDA: distribution plots for all chemical features; correlation heatmap*
- *[ ] Discuss class imbalance: are certain quality scores underrepresented? How does this affect modelling?*

- *[ ] Feature engineering: consider binning quality scores into binary (good/bad) or 3-class (low/medium/high) groups and justify the decision*
- *[ ] Train/test split with stratification to preserve class ratios*
- *[ ] Train **3 classifiers**: Random Forest, Stochastic Gradient Descent (SGD), and Support Vector Classifier (SVC)*
- *[ ] Evaluate each: accuracy, classification report, confusion matrix*
- *[ ] Feature importance chart for the Random Forest model*
- *[ ] Comparison table: summarise the performance of all 3 models side-by-side*
- *[ ] Conclusion: which model is most suitable for deployment and why?*

***Self-Sourcing Guideline:** Search **"Wine Quality dataset UCI"** — it is available on both **Kaggle.com** and the **UCI Machine Learning Repository** (archive.ics.uci.edu/dataset/186/wine+quality) for free. Search **"wine quality prediction Python Random Forest SVM tutorial"** on YouTube. Reference scikit-learn docs for `RandomForestClassifier`, `SGDClassifier`, and `SVC`.*

TASK 3 · Fraud Detection

***Objective:** Build a machine learning pipeline to detect fraudulent financial transactions from a heavily imbalanced dataset, addressing class imbalance as a core challenge.*

***Tech Stack:** Python, pandas, scikit-learn, imbalanced-learn (`SMOTE`), matplotlib, seaborn, Jupyter Notebook*

Feature Checklist:

- *[ ] Load dataset and analyse class imbalance: what percentage of transactions are fraudulent?*
- *[ ] EDA: distribution of transaction amounts for fraud vs. non-fraud; time-of-day analysis*
- *[ ] Discuss why standard accuracy is a **misleading metric** for imbalanced fraud datasets (explain in a markdown cell)*
- *[ ] Apply a class imbalance handling technique: SMOTE oversampling, undersampling, or `class_weight='balanced'`*
- *[ ] Train/test split (ensure fraud cases appear in both splits using stratification)*
- *[ ] Train at least **2 models**: Logistic Regression + one of (Decision Tree, Random Forest, or XGBoost)*
- *[ ] Evaluate using: Precision, Recall, F1-Score, and **AUC-ROC curve** (not just accuracy)*
- *[ ] Explain which metric matters most for fraud detection and why (Recall vs. Precision trade-off)*
- *[ ] Feature importance or coefficient analysis*
- *[ ] Discuss scalability: how would this model handle 1 million transactions per hour?*

Self-Sourcing Guideline:** Search **"credit card fraud detection dataset"** on **Kaggle.com** — the "Credit Card Fraud Detection" dataset (284,807 transactions, 492 fraudulent) is a benchmark dataset specifically designed for this task. Search **"fraud detection Python

imbalanced dataset SMOTE tutorial" *on YouTube. Reference the* `imbalanced-learn` *documentation (imbalanced-learn.org) for SMOTE implementation.*

TASK 4 · Unveiling the Android App Market (Google Play Store Analysis)

Objective: *Perform a comprehensive data analysis of the Google Play Store ecosystem — cleaning messy real-world data, exploring app categories, analysing ratings and pricing trends, and conducting sentiment analysis on user reviews.*

Tech Stack: *Python, pandas, numpy, matplotlib, seaborn,* `TextBlob` *or* `VADER`*, Jupyter Notebook*

Feature Checklist:

- *[ ] Load the Play Store apps dataset and the user reviews dataset separately*
- *[ ] Data cleaning: fix incorrect data types (e.g., "Installs" stored as "10,000+" strings), handle nulls, remove duplicates*
- *[ ] Category analysis: bar chart of app distribution across categories; which categories are most saturated?*
- *[ ] Ratings analysis: distribution of app ratings; average rating by category*
- *[ ] Size and installs analysis: scatter plot of app size vs. number of installs; is there a correlation?*
- *[ ] Pricing analysis: paid vs. free app distribution; price distribution for paid apps; revenue estimate by category*
- *[ ] Sentiment analysis on user reviews: classify reviews as positive/negative/neutral using* `TextBlob` *or* `VADER`
- *[ ] Sentiment by category: which app categories have the most positive/negative user sentiment?*
- *[ ] At least one interactive visualisation using* `plotly` *(optional but strongly recommended)*
- *[ ] Conclusion: 3 data-driven insights for a developer planning to launch a new app*

Self-Sourcing Guideline: *Search* ***"Google Play Store dataset"*** *on* ***Kaggle.com*** *— the dataset "Google Play Store Apps" is publicly available. Search* ***"Google Play Store EDA Python tutorial"*** *on YouTube. For sentiment analysis, search* ***"VADER sentiment analysis Python tutorial"*** *— VADER is pre-built for social media text and requires no training.*

TASK 5 · Autocomplete and Autocorrect Data Analytics

Objective: *Analyse the efficiency and accuracy of autocomplete and autocorrect algorithms using NLP techniques. Implement and compare multiple approaches for text prediction and spelling correction on a real text dataset.*

Tech Stack: *Python, pandas, NLTK,* `pyspellchecker` *or* `textdistance`*,* `collections`*, matplotlib, Jupyter Notebook*

Feature Checklist:

- *[ ] Collect or download a large text corpus (see Self-Sourcing Guideline)*
- *[ ] NLP preprocessing: tokenisation, lowercasing, punctuation removal, stopword removal*
- *[ ]* ***Autocomplete implementation:*** *build a frequency-based n-gram model (bigram or trigram) that predicts the next word given a prefix*
- *[ ] Test autocomplete on at least 10 input prefixes and display top 3 predictions per prefix*
- *[ ]* ***Autocorrect implementation:*** *implement edit-distance based correction using* `pyspellchecker` *or custom Levenshtein distance logic*
- *[ ] Test autocorrect on at least 20 deliberately misspelled words and measure correction accuracy*
- *[ ] Performance metrics: define and calculate precision and recall for both autocomplete and autocorrect*
- *[ ]* ***Algorithm comparison:*** *compare at least 2 approaches (e.g., frequency-based vs. neural n-gram, or two different edit-distance algorithms)*
- *[ ] Visualisation: bar chart of top 20 most frequent words; confusion matrix for autocorrect (correct/incorrect before and after)*
- *[ ] Discussion: what are the limitations of your implementation vs. production systems like Google Keyboard?*

Self-Sourcing Guideline: *For a text corpus, download* ***"Project Gutenberg books"*** *(gutenberg.org — free public domain texts) or search* ***"large text corpus NLP dataset"*** *on* ***Kaggle.com*** *(the "Enron Email Dataset" or "Wikipedia Text Corpus" work well). Search* ***"Python autocomplete n-gram model tutorial"*** *and* ***"Python spell checker pyspellchecker tutorial"*** *on YouTube. Reference the NLTK documentation for* `ngrams` *and* `FreqDist`*.*

🔐 TRACK: Security Analyst

***Completion Rule:** Complete **at least 4** of the 10 tasks below. Tasks are categorised into three difficulty tiers: Beginner (Tasks 1–6), Intermediate (Tasks 7–8), and Advanced (Tasks 9–10). Mix tiers freely — choose any 4 that match your current knowledge level.*

Deliverable Types:

- ***Practical Tasks** (1, 2, 3, 7, 8, 9, 10): Require a GitHub repo with code/scripts/screenshots **and** a demo video.*
- ***Research/Report Tasks** (4, 5, 6, 10): Require a GitHub repo with a well-structured `.md` report file. No demo video required for pure research tasks.*

BEGINNER LEVEL (Tasks 1–6)

TASK 1 · Basic Network Scanning with Nmap

***Objective:** Perform a network scan to identify open ports and services running on a local machine or virtual machine using Nmap, and document your findings with security analysis.*

***Tech Stack / Tools:** Nmap, Linux terminal (or Windows PowerShell), a local VM (e.g., VirtualBox with Kali Linux or Ubuntu)*

Feature Checklist:

- *[ ] Install Nmap on your system (document the installation steps in README.md)*
- *[ ] Perform a **basic scan** (`nmap [target IP]`) and record results*
- *[ ] Perform a **service version scan** (`nmap -sV [target IP]`) to identify running services*
- *[ ] Perform an **OS detection scan** (`nmap -O [target IP]`) and record results*
- *[ ] Identify all open ports found and list the service running on each*
- *[ ] For each open port, write a brief explanation of: what the service does, and whether it poses a security risk*
- *[ ] Document your findings in a structured `nmap_scan_results.txt` file*
- *[ ] Include screenshots of Nmap terminal output in your GitHub repo*
- *[ ] README.md must explain: what Nmap is, why network scanning matters, and ethical use guidelines*

> *⚠️ **Ethics Note:** Only scan machines you own or have explicit permission to scan. Always use a local VM for this task — never scan external or production systems.*

Self-Sourcing Guideline: *Search* ***"Nmap tutorial beginners 2024"*** *on YouTube for a complete walkthrough. Download* ***Kali Linux*** *as a free VirtualBox VM from kali.org/get-kali — it comes with Nmap pre-installed. Reference the official* ***Nmap documentation*** *(nmap.org/docs.html) for flag explanations. Search* ***"common open port security risks"*** *on Google to help write your port analysis.*

TASK 2 · Basic Firewall Configuration with UFW

Objective: *Set up and configure a basic firewall on a Linux system using UFW (Uncomplicated Firewall), applying rules to allow and deny specific types of traffic.*

Tech Stack / Tools: *UFW, Linux (Ubuntu or Kali Linux in a VM)*

Feature Checklist:

- *[ ] Install UFW if not already present:* `sudo apt install ufw`
- *[ ] Enable UFW:* `sudo ufw enable`
- *[ ] Configure rule to* ***allow SSH traffic*** *(port 22):* `sudo ufw allow ssh`
- *[ ] Configure rule to* ***deny HTTP traffic*** *(port 80):* `sudo ufw deny http`
- *[ ] Add at least 2 additional rules of your choice (e.g., allow HTTPS, deny a specific IP range)*
- *[ ] Verify active rules:* `sudo ufw status verbose` *— take a screenshot*
- *[ ] Test that denied traffic is actually blocked (document your testing method)*
- *[ ] Create a* `ufw_configuration.sh` *script that applies all your rules in sequence (runnable script)*
- *[ ] README.md must explain: what a firewall does, what each rule achieves, and why these specific rules were chosen*

Self-Sourcing Guideline: *Search* ***"UFW firewall Ubuntu tutorial"*** *on YouTube for a step-by-step walkthrough. Reference the official* ***UFW documentation*** *(help.ubuntu.com/community/UFW) for all available commands and flags. Search* ***"Linux firewall rules best practices"*** *on Google to inform your rule choices and README analysis.*

TASK 3 · SQL Injection on DVWA (Low Security)

Objective: *Demonstrate a classic SQL Injection vulnerability by exploiting the login form of DVWA (Damn Vulnerable Web Application) on its Low security setting, and document the attack with an explanation of how it works and how to prevent it.*

Tech Stack / Tools: *DVWA (runs on XAMPP/LAMP), a web browser, Burp Suite (optional)*

Feature Checklist:

- *[ ] Install and configure DVWA on a local server (using XAMPP on Windows or LAMP on Linux — document the setup)*
- *[ ] Set DVWA security level to **Low** in the settings*
- *[ ] Navigate to the SQL Injection module*
- *[ ] Perform a basic SQL Injection using a classic payload (e.g., `' OR '1'='1`) on the input field*
- *[ ] Document the output — what data was exposed by the injection?*
- *[ ] Attempt at least **2 different payloads** and document each result*
- *[ ] Capture screenshots of each injection attempt and its output*
- *[ ] README.md must explain in plain language: what SQL Injection is, why this payload works, what data it exposed, and how a developer would fix this vulnerability (parameterised queries / prepared statements)*
- *[ ] Include a `sql_injection_notes.md` file with your payload log and analysis*

> ⚠️ ***Ethics Note:** This task must be performed only on DVWA running locally on your own machine. Never attempt SQL injection on any real website or service.*

***Self-Sourcing Guideline:** Search **"DVWA setup tutorial XAMPP"** on YouTube. Search **"SQL injection DVWA low security tutorial"** for the exploitation walkthrough. Reference **PortSwigger Web Security Academy** (portswigger.net/web-security/sql-injection) for a thorough explanation of SQL injection and how prepared statements prevent it — this is the best free resource available.*

TASK 4 · Research Report: Common Network Security Threats

***Objective:** Write a comprehensive, well-structured research report on common network security threats — covering how each attack works, its real-world impact, and specific mitigation strategies.*

***Tech Stack / Tools:** Markdown (for the report), GitHub (for submission)*

Feature Checklist:

- *[ ] Report saved as `network_security_threats_report.md` in your GitHub repo*
- *[ ] Introduction: why network security threats matter in today's landscape (1 paragraph)*
- *[ ] **DoS/DDoS Attacks:** explanation of how they work, a real-world example (cite a known incident), impact, and 3 specific mitigation strategies*
- *[ ] **Man-in-the-Middle (MITM) Attacks:** explanation, real-world example, impact, and 3 specific mitigations*
- *[ ] **IP Spoofing:** explanation, real-world example, impact, and 3 specific mitigations*
- *[ ] **DNS Poisoning/Spoofing:** explanation, real-world example, impact, and 3 specific mitigations (bonus — include this 4th threat for a stronger submission)*
- *[ ] Comparison table: summarise all threats across: attack vector, who is at risk, difficulty to execute, ease of mitigation*
- *[ ] Conclusion: 3 key takeaways for a network administrator*

- *[ ] References section: cite at least 4 credible sources (NIST, CISA, academic papers, or reputable security publications)*

📹 ***No demo video required for this task.***

Self-Sourcing Guideline: *Use **NIST** (nist.gov), **CISA** (cisa.gov), and **SANS Institute** (sans.org/reading-room) as primary reference sources. Search each threat on **Mitre ATT&CK** (attack.mitre.org) for detailed technical descriptions. For real-world examples, search each attack type on **Wired** or **Krebs on Security** (krebsonsecurity.com).*

TASK 5 · Research Report: Social Engineering Attacks

Objective: *Research and write a detailed report on social engineering attack types — focusing on phishing, pretexting, and baiting — including real-world case studies and prevention recommendations.*

Tech Stack / Tools: *Markdown (for the report), GitHub (for submission)*

Feature Checklist:

- *[ ] Report saved as `social_engineering_report.md` in your GitHub repo*
- *[ ] Introduction: define social engineering and why it is considered one of the most effective attack vectors (cite statistics if available)*
- *[ ] **Phishing:** types (spear phishing, whaling, vishing, smishing), how it works, a documented real-world case study, and 4 prevention recommendations*
- *[ ] **Pretexting:** definition, how an attacker builds a false scenario, a documented case study, and 3 prevention measures*
- *[ ] **Baiting:** physical and digital baiting (infected USB drives, fake downloads), case study, and 3 prevention measures*
- *[ ] **Quid Pro Quo** (bonus): explanation and prevention*
- *[ ] Comparison table: attack type, primary target, psychological lever exploited, and best countermeasure*
- *[ ] Organisational recommendations: a 5-point employee security awareness training checklist*
- *[ ] References section: cite at least 4 credible sources*

📹 ***No demo video required for this task.***

Self-Sourcing Guideline: *Search **"social engineering attacks report"** on **SANS Reading Room** (sans.org/reading-room) for academic-quality papers. Reference the **CISA Social Engineering guide** (cisa.gov) for official guidance. For case studies, search specific attacks (e.g., "2011 RSA SecurID spear phishing attack" or "Twitter 2020 hack social engineering") on **Wired** or **Dark Reading** (darkreading.com).*

TASK 6 · Research Report: The Importance of Patch Management

Objective: *Write a report explaining what patch management is, why unpatched systems represent one of the largest attack surfaces in cybersecurity, and how organisations should implement an effective patching strategy.*

Tech Stack / Tools: *Markdown (for the report), GitHub (for submission)*

Feature Checklist:

- *[ ] Report saved as* `patch_management_report.md` *in your GitHub repo*
- *[ ] Introduction: define patch management and its role in the vulnerability lifecycle*
- *[ ]* ***Why patches matter:*** *explain how vulnerabilities are discovered, reported (CVEs), and exploited — include at least 2 real-world breaches caused by unpatched systems (e.g., WannaCry/EternalBlue, Equifax breach)*
- *[ ]* ***Consequences of not patching:*** *data breaches, ransomware attacks, compliance violations, financial penalties (cite statistics)*
- *[ ]* ***Patch management lifecycle:*** *Discovery → Assessment → Testing → Deployment → Verification — explain each phase*
- *[ ]* ***Best practices:*** *a prioritised 7-step patch management checklist for organisations*
- *[ ]* ***Challenges:*** *why organisations struggle to patch promptly (legacy systems, downtime concerns, testing requirements) and how to overcome each*
- *[ ] References section: cite at least 4 credible sources (NIST, CISA, CVE database)*

📹 ***No demo video required for this task.***

Self-Sourcing Guideline: *Search* ***"NIST patch management guidelines"*** *— the NIST Special Publication* ***800-40*** *is the definitive free guide on this topic (available at nvlpubs.nist.gov). Search* ***"WannaCry ransomware EternalBlue analysis"*** *for the most documented real-world example of patch management failure. Reference the* ***CVE database*** *(cve.mitre.org) to explain how vulnerabilities are catalogued and scored (CVSS scoring).*

INTERMEDIATE LEVEL (Tasks 7–8)

TASK 7 · Vulnerability Scanning with Nikto

Objective: *Use Nikto to perform an automated vulnerability scan on a web server, analyse the results, and document identified security issues with recommended remediation steps.*

Tech Stack / Tools: *Nikto, Linux terminal, a test web server (use DVWA running locally, or set up a simple Apache server in a VM)*

Feature Checklist:

- *[ ] Install Nikto: document the installation process in README.md*
- *[ ] Run a basic Nikto scan against your target:* `nikto -h [target URL]`

- *[ ] Run a scan with output saved to file:* `nikto -h [target] -o nikto_scan_results.txt`
- *[ ] Identify and list all vulnerabilities/issues reported by Nikto in the output*
- *[ ] For each finding, write: what the vulnerability is, why it is a risk, and how to fix it*
- *[ ] Categorise findings by severity (High / Medium / Low / Informational)*
- *[ ] Run a second scan with an SSL check flag if your target supports HTTPS:* `nikto -h [target] -ssl`
- *[ ] Include screenshots of Nikto running and the output*
- *[ ] README.md must explain: what Nikto does, its limitations (it is a noisy scanner — explain what this means), and the difference between Nikto and Nmap*

Self-Sourcing Guideline: *Search* ***"Nikto web vulnerability scanner tutorial"*** *on YouTube. Use* ***DVWA*** *as your scan target — it is intentionally vulnerable, so Nikto will find many issues. Reference the* ***Nikto GitHub repository*** *(github.com/sullo/nikto) for documentation and flag explanations. Search* ***"OWASP Top 10 vulnerabilities"*** *(owasp.org) to cross-reference Nikto findings against the industry-standard list of web vulnerabilities.*

TASK 8 · Capture Network Traffic with Wireshark

Objective: *Capture live network traffic using Wireshark, apply filters to isolate specific protocols, analyse packet contents, and document your findings with security observations.*

Tech Stack / Tools: *Wireshark, a local network interface (can use your own machine's traffic on a test network or VM)*

Feature Checklist:

- *[ ] Install Wireshark: document the process; note any permissions required (e.g., running as administrator)*
- *[ ] Capture at least* ***2 minutes of live traffic*** *on your local network interface*
- *[ ] Apply display filter for* ***HTTP traffic*** *(`http`) and screenshot the filtered results*
- *[ ] Apply display filter for* ***DNS traffic*** *(`dns`) and screenshot the results*
- *[ ] Apply display filter for* ***TCP traffic*** *(`tcp`) and analyse a complete TCP 3-way handshake — screenshot and annotate the SYN, SYN-ACK, ACK sequence*
- *[ ] Export the capture as* `wireshark_capture.pcap` *and include in GitHub repo*
- *[ ] Identify at least one packet that contains unencrypted data (e.g., an HTTP GET request) and document what information is visible*
- *[ ] Explain in README.md: why unencrypted HTTP traffic is dangerous, and how HTTPS prevents this type of eavesdropping*
- *[ ] README.md must include a glossary: define packet, protocol, port, payload, and handshake in your own words*

> ⚠️ ***Ethics Note:*** *Only capture traffic on networks you own or administer. Do not capture traffic on public Wi-Fi, university networks, or any network without explicit authorisation.*

Self-Sourcing Guideline: *Search* ***"Wireshark tutorial beginners 2024"*** *on YouTube for a complete walkthrough. Reference the* ***Wireshark User Guide*** *(wireshark.org/docs/wsug_html_chunked/) for filter syntax. Search* ***"Wireshark TCP three-way handshake tutorial"*** *specifically to understand and annotate the handshake correctly. Search* ***"Wireshark HTTP packet analysis tutorial"*** *for the unencrypted data demonstration.*

ADVANCED LEVEL (Tasks 9–10)

TASK 9 · Exploit a SQL Injection Vulnerability (Advanced)

Objective: *Extend Task 3's beginner SQL injection demonstration into a comprehensive exploitation exercise — using more advanced payloads, attempting to extract database schema information, and producing a detailed vulnerability report with patching recommendations.*

Tech Stack / Tools: *DVWA (Medium or High security setting),* `sqlmap` *(optional automated tool), Burp Suite (optional for intercepting requests), Linux terminal*

Feature Checklist:

- *[ ] Set up DVWA (Medium security setting for an additional challenge vs. Task 3's Low)*
- *[ ] Manually craft payloads to: bypass login authentication, enumerate database tables, extract column names from a target table*
- *[ ] Document each payload used and the exact output returned*
- *[ ] (Bonus) Use* `sqlmap` *in a controlled, local-only environment to automate discovery — compare its output to your manual findings*
- *[ ] Use Burp Suite to intercept and inspect the HTTP request/response during injection (screenshot the request in Burp)*
- *[ ] Produce a structured* `sql_injection_exploit.sh` *script that documents your manual steps as commented commands*
- *[ ]* ***Remediation section:*** *write a developer-facing explanation of: (1) why this vulnerability exists, (2) how to implement parameterised queries in Python/PHP to fix it, with a code example for each*
- *[ ] README.md: executive summary suitable for a non-technical manager explaining the risk level and business impact*

> ⚠️ ***Ethics Note:*** *This task must be performed exclusively on DVWA running locally on your own machine. Never use* `sqlmap` *or any offensive tools on real systems without written authorisation.*

Self-Sourcing Guideline: *Search* ***"sqlmap tutorial DVWA local"*** *on YouTube — use only in a local VM environment. Reference* ***PortSwigger Web Security Academy***

*(portswigger.net/web-security/sql-injection) for advanced payload techniques. Search **"parameterised queries Python prevention SQL injection"** to write your remediation section. Reference **OWASP SQL Injection Prevention Cheat Sheet** (cheatsheetseries.owasp.org) for the definitive developer guidance.*

TASK 10 · Full Network Security Assessment Report

***Objective:** Conduct a structured, end-to-end security assessment of a local test network using multiple tools, and produce a professional security report suitable for presentation to a technical team or management.*

***Tech Stack / Tools:** Nmap, Wireshark, Nikto (where applicable), Markdown for the report*

Feature Checklist:

- *[ ] Define the scope of your assessment in writing: which IP ranges, which services, and which time window*
- *[ ] **Phase 1 — Reconnaissance:** Run an Nmap scan (`nmap -sV -O [target range]`); document all discovered hosts, open ports, and services*
- *[ ] **Phase 2 — Traffic Analysis:** Capture 5+ minutes of network traffic with Wireshark; filter and analyse HTTP, DNS, and ARP traffic; document any unencrypted sensitive data observed*
- *[ ] **Phase 3 — Web Vulnerability Scan** (if a web server is present): Run Nikto against any identified web server; document findings*
- *[ ] **Findings register:** produce a table listing every finding with columns: Finding ID, Description, Severity (Critical/High/Medium/Low/Info), Affected Asset, Recommended Fix*
- *[ ] **Executive Summary:** a 1-page non-technical summary of the overall risk posture, suitable for a manager*
- *[ ] **Technical Report:** detailed findings for each phase, with screenshots as evidence*
- *[ ] **Remediation roadmap:** prioritise all findings and provide a recommended fix order with effort estimates (Easy/Medium/Hard)*
- *[ ] All files committed to GitHub: `network_security_assessment.md`, `nmap_results.txt`, `wireshark_capture.pcap`*

***Self-Sourcing Guideline:** Search **"network security assessment methodology"** on Google — reference the **OWASP Testing Guide** (owasp.org/www-project-web-security-testing-guide/) and the **PTES Technical Guidelines** (pentest-standard.org) for industry-standard assessment frameworks. Search **"how to write a penetration test report"** on YouTube for guidance on professional report structure. Reference the **CVSS scoring system** (first.org/cvss) to assign consistent severity ratings to your findings.*

CONTACT & SUPPORT

For queries during your internship, reach out through the following channels:

Channel	Details
Email	*support@oasisinfobyte.com*
Telegram Support	*https://t.me/oasisinfobyte (link shared in welcome email)*
LinkedIn	*https://in.linkedin.com/company/oasis-infobyte*
Website	*www.oasisinfobyte.com*

The purpose of this internship is to learn and grow. The tasks may seem easy or difficult — we expect professional regard for every one of them. We don't want to dictate you. It is up to you to seek guidance. This internship will help you enhance your employability prospects and raise your confidence.

— Oasis Infobyte Team
